



PRODUCT BUILD PLAN • SOLO FOUNDER EDITION

Building an AI Interior Design Editor

A complete, phase by phase plan for building a professional 3D interior design tool on the web. Written to be followed by someone who has never written code, while staying technically precise enough to actually build from.



Scope: The editor first. Room scanning is deliberately deferred to a later stage.

Format: Nine phases, each with technologies, methods, and a clear finish line.

What is inside

Part 1 **The product in plain language**

What you are building, who it is for, and why it can win.

Part 2 **The vocabulary you need**

Every technical word in this document, explained once, simply.

Part 3 **How the whole thing fits together**

The architecture as a simple mental picture, and the one rule that makes it buildable.

Part 4 **The nine phases**

Phase 0 Foundation · 1 Room · 2 Furniture · 3 Light and Materials · 4 Catalog · 5 Views · 6 Projects · 7 The AI Assistant · 8 Hero Render

Part 5 **Time, effort, and the honest reality**

How long each stage takes for one person, and what to build first.

Part 6 **Rules that hold across every phase**

Part 7 **How to actually build it**

Part 1 The product in plain language

Before any technology, be clear about the thing itself.

You are building a **web application** that professional interior designers open in a browser. A designer starts with an empty room set to real dimensions, then furnishes and decorates it: dropping in sofas, tables, and lamps from a catalog, changing wall colours and floor finishes, placing lights, and viewing the space as a realistic 3D scene, a clean floor plan, or a simplified model. On top of the manual tools sits an assistant you can talk to in plain English, such as "make this wall sage green and add a warm reading lamp by the armchair," and it makes the change for you.

Later, once the editor is solid, you will add **room capture** so a designer can start from a scan of a client's actual room instead of a blank box. That capture step is the eventual differentiator, but it is intentionally not part of this plan. The editor has to be excellent first, because a scan that drops into a weak editor is worth nothing.

Who this is for and why it can win

The tools designers use today split into two camps. Light consumer apps like Planner 5D are easy but shallow: limited catalogs, weak rendering, no real precision. Heavy professional renderers like D5 and Enscape are powerful but start from architectural drawings and have steep learning curves. Your opening is the middle: a tool that is genuinely easy, looks professional, understands room dimensions, and eventually begins from a real scan. You are not trying to beat the giants at their own game. You are serving the designer who finds the consumer tools too limiting and the professional tools too heavy.

The rest of this document is the construction plan for that editor, broken into nine phases. Each phase builds one working capability, names the exact technologies to use and explains what each one is, describes the method, and ends with a precise finish line: what has to be true for the phase to be done, and what you personally will be able to do once it is.

Part 2 The vocabulary you need

These words appear throughout. Read this once and the rest becomes easy.

The web application itself

Web app. Software that runs inside a browser tab instead of being installed. Your whole product is one of these.

React. The most widely used toolkit for building the visible parts of a web app: buttons, panels, menus, the layout on screen. Think of it as the system that draws and updates your interface as the user clicks around.

TypeScript. The programming language you will write in. It is the popular JavaScript language with an added safety net that catches whole classes of mistakes before they ever run. For a solo builder, that safety net saves days.

Vite. The tool that bundles your code and runs it while you develop, refreshing the screen instantly as you make changes. Plumbing, but essential plumbing.

The 3D part

three.js. The engine that draws 3D graphics inside a browser: rooms, furniture, lights, shadows. Almost every 3D website you have seen uses it. It is the heart of your product.

React Three Fiber (R3F). A bridge that lets you describe a 3D scene using the same React style you use for the interface, instead of controlling three.js by hand. It keeps your 3D world and your buttons speaking the same language.

drei. A box of ready made helpers that sit on top of React Three Fiber: a camera you can orbit, the drag handles for moving objects, environment lighting, model loading. It saves you from rebuilding common pieces.

Scene graph. The single most important idea in this document. It is your master list of everything in the room: every wall, every chair, every light, described as plain data (this chair is here, rotated this much, in this fabric). The 3D picture on screen is generated *from* this list. The list is the truth; the picture is just a view of it.

GLB. The standard file format for a single 3D model, such as one sofa. One GLB file holds the shape and the surface appearance together. Your entire furniture catalog will be GLB files.

PBR material. Short for physically based rendering. It is a realistic description of a surface: its colour, how rough or glossy it is, how metallic it is. PBR is why a wood floor and a chrome lamp catch light differently and look believable.

Behaviour and data

Zustand. A small, simple system for holding your app's live information (the scene graph, what is selected) in one place that every part of the app can read. It is where your master list lives while the app is running.

Undo and redo (the command pattern). A way of building edits so every action can be reversed. Instead of changing the room directly, each action is recorded as a step that knows how to undo itself. Designers expect flawless undo, and this is how you deliver it.

Rendering. Turning the 3D scene into an image. You will have two kinds: *real time* rendering, the live picture that

updates as the designer works, and a slower *hero* render, a single magazine quality image for the client.

Large language model (LLM). The kind of AI that understands and produces language, the technology behind chat assistants. You will use one so designers can edit the room by typing what they want.

Backend. The part of the product that runs on a server rather than in the browser: storing projects, user accounts, the furniture catalog. The browser is the front, the backend is the back.

Part 3 How the whole thing fits together

One picture to hold in your head for the entire build.

Picture five pieces. In the centre is the **master list**, the scene graph: the plain data description of the room and everything in it. Around it sit four things that all talk to that list.

1. The **editor** is what the designer sees and touches. When they drag a chair, it does not move the chair directly. It updates the master list, and the 3D picture redraws to match.
2. The **catalog** is the library of furniture models the designer pulls from. Choosing a sofa adds an entry to the master list pointing at that sofa's file.
3. The **AI assistant** reads the master list to understand the room, then makes the exact same kind of updates the editor's buttons make. It has no special powers. It just presses your buttons through language.
4. The **render service** takes the master list and produces the final polished image.

THE ONE RULE THAT MAKES THIS BUILDABLE

Nothing ever changes the room directly. Every change goes through the master list, and every change is recorded as a reversible step. This single discipline is why undo works everywhere, why saving a project is trivial, and why the AI can be added later without touching the core. Break this rule to move fast early, and you will rewrite the whole thing later. Every phase in this plan respects it.

How to build, so you do not waste months

Build in thin vertical slices, not thick horizontal layers. The tempting mistake is to build a complete master list system, then a complete catalog, then a complete lighting system, and only discover at the very end that the core feel of moving objects is wrong. Instead, get the thinnest possible end to end version working first: an empty room, one piece of furniture you can place and move, one light. Prove the whole loop, then deepen each piece. The phases are ordered to make this happen, and to face the single genuinely hard unknown, whether realistic lighting is achievable in a browser, on day one rather than after months of work.

In plain terms

Before building any feature, you build the skeleton of the whole app and prove the single hardest thing an editor must do: let a person select an object in 3D and move, turn, and resize it smoothly. This phase is also where you learn 3D graphics, which is the only genuinely new skill in the entire project. It is the longest phase and the one to slow down on, because everything else is built on top of what you decide here.

Why this phase matters more than any other

Two things decided here are load bearing for the entire product: the master list (how you describe the room as data) and undo (how every change can be reversed). Get these right and every later phase clicks into place. Get them wrong and you will rebuild the app from scratch six months in. There is no shortcut past this, so it comes first and it gets the most care.

What you will build

A working shell where you can drop a single 3D object into empty space, click to select it, move and rotate and resize it with on screen handles, undo and redo any change, and save the scene to a file and load it back exactly as it was. No room yet, no catalog, no polish. Just the spine, proven end to end.

Suggested technologies

React + TypeScript + Vite. The foundation of the app and the language you write in. Standard, well documented, easy to get help with.

React Three Fiber + drei. Your 3D layer and its helper kit. R3F draws the scene; drei gives you the orbit camera and the move/rotate/resize handles without building them yourself.

Zustand. Holds the master list and the current selection in one place. Chosen over heavier options because it is simple and only redraws the parts of the screen that actually changed, which keeps a busy 3D app fast.

Tailwind CSS. A fast way to style the interface panels and buttons around the 3D view.

immer. A small helper that makes recording reversible changes to the master list straightforward, which is the backbone of undo.

What you need to learn, in order

You do not need to learn everything about 3D. You need four things, and you learn each one at the moment you need it, by building a tiny working example rather than reading theory.

1. A little 3D maths. Just three ideas: how an object's position, rotation, and size combine; why rotations are stored as something called a quaternion instead of simple angles (it avoids a class of glitches); and how a click on screen is turned into a line poked into the 3D world to find what you hit, called raycasting. That last one is how selection works.

2. How three.js thinks. The scene, objects, cameras, lights, and the loop that redraws sixty times a second. Learn this by building two or three plain 3D scenes before adding React on top, so you understand the engine underneath the convenience layer.

3. The React Three Fiber mindset. One key shift trips up newcomers: never drive smooth motion through the normal interface update system, because it is too slow for sixty frames a second; you nudge the 3D object directly instead. Read the R3F performance notes early.

4. Editor architecture. The master list as your own data, kept separate from what three.js draws, plus the reversible step pattern for undo. This is the most valuable thing to study, and the official three.js editor's source code is the best example to learn it from.

The method: learn by building, in small steps

Do not try to build the whole shell at once. Walk these steps, and do not move on until each one visibly works. The first few are throwaway practice; do not polish them.

1. Set up the project and get a blank 3D canvas on screen.
2. In plain three.js, put a spinning cube on screen with a light and a camera you can orbit. This teaches the render loop.
3. Add a few cubes and make clicking one highlight it. This teaches selection by raycasting.
4. Nest objects together and move the parent so the children follow. This is your master list in miniature.
5. Rebuild the click to select scene using React Three Fiber, so you learn the bridge.
6. **Lighting spike (do this now).** In a throwaway scene, load one real furniture model, light it with an environment image and a soft light, and turn on shadows. Judge honestly whether the browser can look good enough for professionals. This answers your biggest risk before you invest in architecture.
7. Build the real master list as your own data, held in Zustand, and draw the scene from it so that adding an entry to the list makes an object appear.
8. Wire selection and the move/rotate/resize handle to the list, so dragging updates the data.
9. Add undo and redo, so every change can be reversed.
10. Add save and load: write the master list to a file and rebuild the scene from it.

This phase is complete when

You can drop a real furniture model into empty space, click to select it, move and rotate and resize it with handles, undo and redo those changes, save the scene to a file, reload the page, load the file, and see the exact same scene restored. And the lighting spike has given you a clear yes or no on browser fidelity.

By the end you can

Manipulate objects in a 3D space entirely through your own master list, with flawless undo and reliable saving. You have proven the spine that every other phase depends on, and you have answered whether the whole approach can look professional.

Watch out

Slow down hardest on two steps: the master list design and the undo system. These are the two things every later phase inherits, and they are painful to change once features are stacked on top. Also, resist building any room or catalog features here. If you find yourself decorating, you have left Phase 0 too early.

In plain terms

Now you build the container: a room with real walls, a floor, and a ceiling, set to dimensions the designer types in or draws. This is the stage on which everything else is placed.

What you will build

Start with the simplest useful version: a rectangular room where the designer sets width, length, and height, and the walls, floor, and ceiling appear at the right size. Then add a simple drawing step where the designer clicks points to trace an outline of any shape and the walls rise from that outline. The walls must be treated as individual surfaces, because later the designer will recolour and retexture each one. Show real measurements as they build.

Suggested technologies

React Three Fiber, continued. The room is just more objects in your master list: wall entries, a floor entry, a ceiling entry.

Generated shapes. Rather than downloading a room model, you build the walls from the dimensions in code, so they can be edited freely. `three.js` has the tools to turn an outline into solid walls.

A 2D drawing surface for the floor plan step. A simple flat canvas where the designer clicks to place corners, which you then raise into 3D walls.

Real world units. Everything measured in metres from the very start, so a three metre wall is genuinely three metres. This consistency matters enormously later.

The method

Ship the rectangular room first and make sure the camera can move through it comfortably. Only then add the freeform outline drawer. Treat doors and windows as openings you can add to a wall, and it is fine to leave those for a small follow up once the basic room works.

This phase is complete when

A designer can create a room of their own dimensions, either as a box or by drawing an outline, and move the camera through it as a believable space with correct measurements.

By the end you can

Produce the actual space a design will live in, sized to the real world, ready to be furnished and decorated.

In plain terms

This is where the product becomes real. The designer places furniture in the room and arranges it: a sofa that sits on the floor at the right size, a table they can nudge and turn, pieces that do not pass through each other. If you get nothing else perfect, get this loop to feel natural.

What you will build

Loading of real furniture models that are already sized to real life, placement that drops a piece onto the floor at correct scale, and the move, rotate, and resize handles from Phase 0 now applied to furniture. Add gentle snapping so pieces settle against the floor and align to walls, and detection so two pieces flag or resist overlapping. Add duplicate, delete, and selecting several pieces at once. Start with just three to five models kept in the app itself; the full catalog comes in Phase 4.

Suggested technologies

GLB models loaded through drei. Each furniture piece is a GLB file. drei's loader brings it into the scene as an entry in your master list.

Rapier, or simple box checks. Rapier is a physics helper that can detect when pieces collide. For a first version, comparing the invisible boxes around each object is simpler and often enough. Start simple, upgrade only if needed.

Snapping logic you write. Small rules that pull a piece down to the floor and align it to a nearby wall, so placement feels assisted rather than fiddly.

The method

Reuse the Phase 0 handles rather than building new ones. Everything here should flow through the master list and the undo system, so placing, moving, and deleting furniture are all reversible without any extra work. Insist that every furniture model is sized to real metres before it enters the scene, or arrangement becomes chaos.

This phase is complete when

You can furnish an empty room with several pieces that sit correctly on the floor, align sensibly to walls, do not clip through each other, and can be duplicated, deleted, and undone cleanly.

By the end you can

Furnish and rearrange a room. This is the moment the product genuinely exists. Everything after this deepens it rather than creating it.

Watch out

Performance starts to matter here. A fully furnished room is many objects at once. Use the standard techniques for drawing repeated items efficiently and hiding what the camera cannot see, and simpler versions of distant models. Building this habit now prevents a sluggish app later.

In plain terms

Now the designer can change how the room looks: add and place lights, and change the colours and finishes of walls, floors, and furniture. These are the two features you named as the point of the product, so this is where fidelity has to shine.

What you will build

A base lighting setup that makes the room look good immediately, plus lights the designer can place, each with adjustable brightness, colour, and warmth, casting believable shadows. And a materials system: click a surface and recolour it, swap its finish from a library (oak, marble, sage plaster), and adjust how glossy or matte it is, with fabric and finish options for furniture too.

Suggested technologies

Environment lighting from an HDRI. An HDRI is a special panoramic photo of a real place's light. Using one to light your scene is the single biggest trick for making 3D look real, because it bathes everything in believable, directional light.

Poly Haven. A free, high quality source of both HDRI lighting images and PBR surface finishes. Your starting library for materials and lighting comes from here at no cost.

Placeable lights in your master list. Point lights, spotlights, and soft area lights, each an entry the designer can add, move, and tune, just like furniture.

Real time shadows. Turned on and tuned carefully, because shadows are what sell the realism, and untuned shadows are what break it.

PBR materials. The realistic surface descriptions, so recolouring a wall or changing a floor to oak reacts correctly to the lighting.

The method

This is the phase to invest real time in tuning, because it sets your quality bar. Get the environment lighting and shadows looking excellent on a furnished room before moving on. Colour management matters here: configure the scene so colours look accurate rather than washed out, a setting `three.js` supports but that must be turned on deliberately.

This phase is complete when

You can change a wall to sage green, swap the floor to oak, and add a warm floor lamp that casts a believable shadow, all looking convincingly real.

By the end you can

Deliver the two signature editing tools, lighting and materials, at a quality a professional would accept. This is the milestone worth showing publicly for the first time.

In plain terms

Until now furniture was a few models baked into the app. Now you build a real, browsable library the designer searches and pulls from, stored properly on a server and delivered quickly.

What you will build

A catalog browser with search, categories, and thumbnails. A backend that stores the furniture files and their details. An automatic step that shrinks and optimises each model so it loads fast. And a way to fill the catalog: turning furniture photos into 3D models using AI, alongside curated models from marketplaces.

Suggested technologies

PostgreSQL. A reliable database that holds each item's details: name, category, dimensions, and where its file lives.

Object storage plus a CDN. Object storage (such as Amazon S3 or Cloudflare R2) holds the actual model files. A CDN is a delivery network that serves those files quickly to users anywhere. Together they make the catalog fast and cheap to run.

glTF-transform. A tool that automatically compresses models and their textures and makes simpler versions for distant viewing, so a heavy model becomes light without you editing it by hand.

Image to 3D generators (Tripo, 3D AI Studio). AI services that turn a single furniture photo into a usable 3D model complete with realistic surfaces, in minutes. This is how you fill a catalog quickly instead of modelling or licensing everything. Treat generated pieces as a fast starting library, not flawless final assets.

Marketplaces (Sketchfab and similar). Sources of curated, higher quality models to supplement the generated ones.

The method

Do not build this before the placement loop of Phase 2 works, or you will have a beautiful catalog feeding a broken editor. Enforce real world sizing as every model enters the library, so nothing arrives at the wrong scale. Seed the catalog in one coherent style at a time (a matching sofa, table, lamp, and rug) so early rooms look designed rather than random.

This phase is complete when

A designer can browse a real catalog, search it, and drag pieces into the room that arrive correctly sized and load quickly.

By the end you can

Offer a genuine library of furniture, which is what turns a demo into something a designer could actually use for real projects.

In plain terms

The same room, shown different ways with one click: a realistic view, a simple solid colour model for studying shape, a line drawing, and a clean top down floor plan with measurements. Designers present to clients in different styles, and offering all of them in one tool is a real point of difference.

What you will build

A view switcher that flips the whole scene between realistic, matte solid, outline, and a measured top down floor plan generated from the room's data. Optionally a hand drawn sketch look.

Suggested technologies

Material overrides. Most view modes are achieved by temporarily replacing every surface with a plain or outline appearance, which three.js supports directly.

An orthographic camera. A camera with no perspective distortion, which is exactly what a proper top down floor plan needs.

A plan generator from your master list. Because the room and furniture are plain data, you can draw a clean 2D floor plan with labels and measurements directly from that data.

This phase is complete when

One click switches cleanly between realistic, matte, outline, and a measured floor plan view of the same room.

By the end you can

Present a design in several professional styles from one tool, a feature most competitors do not offer together.

Why this is easier than it sounds

Because you built everything on a plain data master list from Phase 0, these views are mostly presentation changes over data you already have, not new machinery. This is a direct payoff of the discipline in the early phases.

6

Projects, Saving, and Sharing

TURNING THE TOOL INTO A PRODUCT

In plain terms

Make it something designers can log into, save their work in, come back to, and send to clients. This is the difference between an impressive demo and a product people rely on.

What you will build

User accounts and sign in, a list of saved projects, full saving and loading of a design to the server, automatic saving so work is never lost, and a read only link a designer can send a client to view the design in the browser without an account.

Suggested technologies

Clerk or Supabase. Services that handle user accounts and secure sign in for you, so you do not build sensitive login machinery from scratch.

Your database and storage from Phase 4. Projects are saved as the master list data, which is already designed to be saved and reloaded.

A read only viewer. The same 3D scene shown in a look but do not touch mode, behind a shareable link.

This phase is complete when

A designer can sign in, save a project, reload it later exactly as they left it, and send a client a link that opens the design to view.

By the end you can

Offer a real product with accounts, saved work, and client sharing, the foundation for actually charging for it.

In plain terms

The designer types what they want, in plain English, and it happens: "make this wall sage green," "add a warm reading lamp by the armchair," "furnish this empty room in a warm minimalist style." The assistant understands the room, understands the request, and makes the change. This is the feature that makes the tool feel modern and fast.

THE GOLDEN RULE OF THE AI LAYER

The AI never changes the room directly. It only presses the same buttons the designer's own tools press.

Every AI edit flows through the master list and the undo system built in Phase 0. This one constraint is what makes the assistant trustworthy: every change it makes can be reversed or adjusted by hand, it can never corrupt the design, and you can show the designer exactly what it is about to do before it does it. An AI editor built any other way becomes a slot machine professionals will not trust.

How "type what you want" actually works

The assistant is a language AI given a specific set of actions it is allowed to take, matching your editor's own actions: recolour a surface, place an item, add a light, move something, remove something, search the catalog. Its whole job is to turn a sentence into the right sequence of those actions. The pipeline has a few parts, and two of them are where the real craft lives.

1. **A summary of the room for the AI.** You never hand it the raw 3D data. You give it a compact written description: what objects are in the room, where they are, their sizes, the wall the designer is looking at, and what is currently selected. That is what it reasons about.
2. **The list of allowed actions.** A small, fixed menu of things it can do, each mapping to one of your editor's reversible steps.
3. **Working out what the words point to (the hard part).** "This wall" means the one selected or looked at. "The armchair" means matching it by name, and asking which one if there are two. Phrases like "by the armchair," "in the left corner," or "facing the television" require real spatial understanding. You must not let the AI guess coordinates for these. Instead you build a small set of trusted spatial helpers (next to, in the corner, facing, against the wall) that calculate the exact spot, and the AI simply chooses which helper to use. This spatial resolver is the single thing that decides whether the assistant feels magical or clumsy.
4. **Grounding vague descriptions to real furniture (the other hard part).** "A warm mid century reading lamp" is not a specific catalog item. You match the description against your catalog using a search that understands meaning and images, offer the AI the best candidates, and let it pick. This connects loose language to real products you actually have.
5. **Finalising placement precisely.** The AI decides the intent and rough location; a small automatic solver then sets the exact position, on the floor, with proper clearance, not overlapping anything, facing sensibly. Language AIs are poor at producing physically correct positions, so this step is handled by reliable logic, not by the AI.
6. **Doing it, checking it, and fixing it.** The chosen actions run as one reversible bundle. The result is checked, and if something is off, it is nudged into place or the AI is asked to correct itself. The designer is shown the plan and a preview before anything is committed.

Auto furnish and layout suggestions

For requests like "furnish this room" or "suggest a layout," build in three layers, in order. First, **rules plus a solver**: encode real design guidelines (walkway widths, a focal point, sensible seating, clear paths) and let reliable logic arrange furniture to satisfy them. This is dependable and controllable, and it should be the core. Second, an **AI proposal on top**: the language AI suggests an arrangement given the room and the brief, and the solver corrects it into something physically valid, giving you creativity plus correctness. Third, much later, **learned arrangement models** from current research for more variety, which is a large undertaking and the one place your machine learning background could build a genuine advantage, but not where to begin.

Suggested technologies

A large language model with tool use. A capable chat AI that can call your defined actions. You can route simple requests to a cheaper, faster model and complex multi step requests to a stronger one, to control cost.

A meaning based search (embeddings) for the catalog. The technology that lets "warm reading lamp" find the right products by understanding the description rather than matching exact words.

A small placement solver you write. The reliable logic that turns rough intent into exact, valid positions.

Your Phase 0 action and undo system. The assistant plugs directly into it. This is why the AI can be added now without disturbing anything already built.

This phase is complete when

A request like "make this wall sage green and add a reading lamp by the armchair" is understood, carried out correctly, and can be undone in a single step, and "furnish this room" produces a sensible, valid arrangement.

By the end you can

Let designers edit and furnish rooms by simply describing what they want, with every AI action remaining fully reversible and under their control.

Watch out

The easy majority of this phase is the standard assistant loop. The parts that decide whether it delights or frustrates are the spatial resolver and the catalog matching. Put your effort there. And keep the assistant strictly additive: the day it can change the design outside the reversible system is the day professionals stop trusting it.

In plain terms

The live view in the editor is good, but for a client presentation a designer wants a single, stunning, magazine quality image. This phase produces that final polished picture from the current design.

What you will build

One or both of two paths. A fast path where the live scene is turned into a photorealistic image by an AI in seconds, guided so it stays faithful to the actual design rather than inventing a different room. And a slower, exact path that produces a truly physically accurate image using professional rendering, for final deliverables where precision is essential.

Suggested technologies

AI image enhancement guided by the scene. You take the live picture along with hidden data about the scene's depth and the identity of each object, and use an image AI to make it photorealistic. The guidance is what keeps the sofa the same sofa and the layout the same layout, rather than a pretty but wrong picture. This is fast and inexpensive and sits squarely in your area of strength.

Headless Blender (Cycles) on a rented GPU. Blender is professional grade 3D software; Cycles is its physically accurate renderer. Run automatically on a rented graphics computer (from services that rent them by the hour), it produces true to life images. Slower and pricier, for final quality.

A job queue. Because these renders take time, requests wait in a line and process in the background while the designer keeps working.

This phase is complete when

A designer can turn the current design into a polished, client ready image.

By the end you can

Deliver the final presentation image that wins a designer their client, closing the loop from empty room to finished pitch.

Part 5 Time, effort, and the honest reality

What this actually takes for one person.

These are estimates of focused, full time effort for someone strong in web and backend work but new to 3D, using an AI coding assistant to speed up the routine parts. They assume you are learning 3D as you go, which is the biggest single variable.

Milestone	What you have	Full time effort
Core editor (Phases 0 to 3)	A furnished room with real lighting and material editing that looks professional	About 3 to 4 months, with Phase 0 alone a third of that
Usable product (through Phase 6, plus first AI)	Catalog, accounts, saving, sharing, and basic assistant	About 6 to 8 months total
Full vision (all phases, plus later room capture and polish)	The complete product	About 12 to 18 months total

Those are full time figures. If you build this alongside other commitments at a part time pace, a realistic rule is to roughly double the calendar time, and to account for the fact that switching in and out of a complex codebase makes part time hours less productive per hour. A usable product then lands closer to well over a year of calendar time, and the full vision becomes a multi year commitment.

What the AI coding assistant does and does not speed up
It genuinely accelerates the routine building, the standard interface and backend work, roughly half the total. It does not shorten the three things that dominate this project: learning 3D, which happens in your own head; the taste and repeated refinement that make a tool feel professional, which is human judgement; and filling the catalog, which is sourcing and curation, not coding.

THE SINGLE MOST IMPORTANT PIECE OF ADVICE

Do not aim at the full product first. Aim at the end of Phase 3. A furnished room with real lighting and material editing is the cheapest way to put something in front of real interior designers and find out whether they want this. If they do, the long build is justified. If they shrug, you have saved yourself a year or more. Build the proof, then let real users decide whether the rest earns its time.

Part 6 Rules that hold across every phase

Four disciplines that keep the whole thing sound.

1. Real world scale is never optional

Every furniture model must be sized to real metres before it enters a room. The moment a chair arrives at the wrong scale, arrangement, measurements, and the floor plan all break. Enforce this at the point every model is added, without exception.

2. The master list is the only source of truth

Nothing about the room lives anywhere except the master list. The 3D picture is always generated from that list, never edited independently. This is what keeps saving, loading, undo, the AI, and the different views all consistent with each other.

3. Undo stays working, forever

Every new feature that changes the room must go through the reversible step system. The instant a feature quietly changes something outside that system, undo becomes unreliable, and unreliable undo is something professionals will not forgive.

4. Performance is a habit, not a final step

From the moment rooms have furniture, keep drawing efficient: reuse repeated objects, hide what the camera cannot see, and use simpler versions of distant models. A tool that stutters feels broken regardless of how good it looks in a screenshot.

The sequencing logic, in one paragraph

Phases 0 to 3 build an editor that feels real and prove that a browser can look professional. Phases 4 to 6 turn that into something a designer can actually use, fill with real furniture, and save. Phases 7 and 8 are the multipliers that make it fast and beautiful. If you want a first public milestone, the end of Phase 3 is the thing worth showing the world, and the end of Phase 2 is the thing worth showing yourself.

Part 7 How to actually build it

You do not need to become a programmer to get this built.

Each phase in this plan is a self contained chunk of work with a clear finish line. That structure exists for a reason: it is exactly how you hand work to an AI coding assistant, one phase at a time. You describe the phase, its technologies, and its finish line; the assistant writes the code; you test against the finish line before moving on. The phases are ordered so each one gives the next a working foundation to build on.

Spend the most care specifying **Phase 0**, because every later phase inherits its decisions. Get the master list and the undo system right, run the lighting test early to confirm the whole approach can look professional, and only then proceed. The later phases are more forgiving.

A sensible order for the first month

Learn the small amount of 3D by building the tiny practice examples in Phase 0. Run the lighting test to answer the fidelity question. Then build the master list and the undo system properly. That single month tells you almost everything about whether this project will be smooth or hard, because the only truly new skill, 3D, is concentrated right there. Everything after it is construction you can direct with confidence.

This document is a plan, not a contract. Treat the phase order as firm, because it is built around dependencies and around facing the hardest unknown first. Treat the specific tools as strong recommendations that you can swap as you learn, as long as you keep the four standing rules intact.